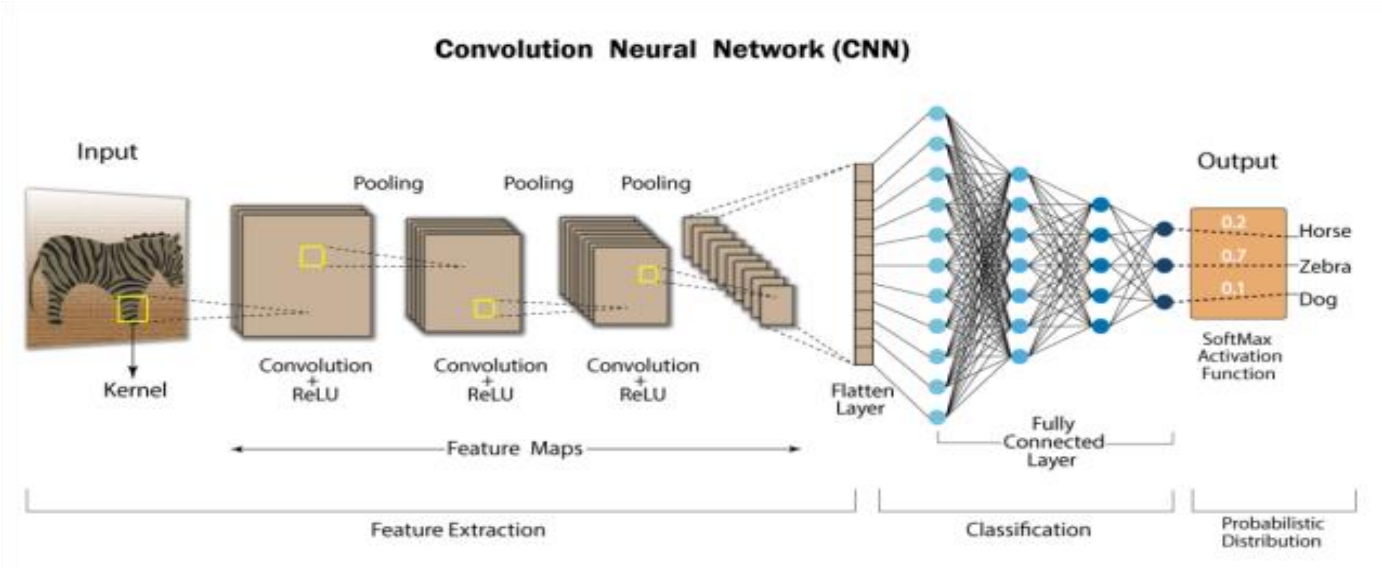


Deep Learning Model for Binary Animal Classification

-ABHISHEK RAWAT

ABSTRACT

This project focuses on developing a Convolutional Neural Network (CNN) to classify images of horses and zebras using the Horse2Zebra dataset. The system preprocesses images, applies data augmentation to improve generalization, and trains a deep learning model capable of binary classification. Techniques such as batch normalization, dropout regularization, early stopping, and learning rate scheduling are incorporated to enhance training stability and model performance. The model's accuracy and reliability are evaluated using validation metrics, classification reports, and confusion matrices. The project demonstrates the effectiveness of CNNs in solving real-world image recognition problems and highlights the potential of deep learning in animal species classification.

INTRODUCTION

Image classification plays a crucial role in computer vision, enabling machines to understand and categorize visual information. Deep learning, particularly Convolutional Neural Networks (CNNs), has significantly advanced the ability to classify complex image data accurately. CNNs automatically learn hierarchical features from images, making them ideal for tasks involving pattern recognition.

This project applies CNN-based deep learning techniques to classify two visually similar animal species: **horses and zebras**. Although they share similar body structures, their patterns differ significantly, providing an interesting challenge for automated classification systems. Using the publicly available Horse2Zebra dataset, this project demonstrates the process of building, training, and evaluating a CNN model capable of distinguishing between these two categories.

Problem Statement

The problem addressed in this project is:
How can a deep learning model be developed to accurately classify horse and zebra images using a CNN-based architecture, despite their visual similarities and dataset challenges?

To solve this, an end-to-end system is developed that includes data preprocessing, augmentation, model training, optimization, and performance evaluation.

Methodology

The methodology followed in this project is divided into key stages:

---

1. Dataset Acquisition

- The **Horse2Zebra** dataset from TensorFlow Datasets (TFDS) is used.
- trainA contains horse images, and trainB contains zebra images.
- Each image is paired with a label (0 = Horse, 1 = Zebra).

---

2. Data Preprocessing

- Images are resized to **128 × 128** pixels to standardize input size.
- Pixel values are scaled to the range **[0, 1]** to improve learning stability.
- Data is converted into TensorFlow datasets for efficient loading and batching.

---

3. Data Augmentation

To reduce overfitting and improve model generalization, the following transformations are applied:

- Random horizontal flip
- Random rotation
- Random zoom

- Random contrast adjustment

This increases the variability of training samples without collecting new data.

---

4. Dataset Splitting

- Combined dataset is shuffled and split into:
  - **80% training data**
  - **20% validation data**
- Both sets are batched and prefetched for performance optimization.

---

5. Model Architecture (CNN Classifier)

A deep CNN model is constructed with:

- Multiple **Conv2D** layers for feature extraction
- **Batch Normalization** to stabilize training
- **MaxPooling** layers for downsampling
- **Flatten** + **Dense** layers for classification
- **Dropout** (0.5) to prevent overfitting
- **Sigmoid output activation** for binary classification

---

6. Model Compilation

Compiled with:

- Optimizer: **Adam**
- Loss function: **Binary Crossentropy**
- Evaluation metric: **Accuracy**

---

7. Training with Callbacks

To improve performance:

- **ReduceLROnPlateau** lowers the learning rate when validation loss stagnates

- **EarlyStopping** prevents overfitting by halting training early
- Best weights are automatically restored

---

8. Model Evaluation

The model is evaluated using:

- Training vs validation accuracy/loss curves
- Classification Report (Precision, Recall, F1-score)
- Confusion Matrix
- Visual predictions on sample images

Objective

The objective of this project is to design and implement a deep learning-based image classification system that differentiates between horse and zebra images using the horse2zebra dataset. The project focuses on data preprocessing, augmentation, model development using Convolutional Neural Networks (CNNs), and performance optimization through techniques such as batch normalization, dropout, and learning rate scheduling. The model is trained, validated, and evaluated to measure its effectiveness in classifying animal species with high accuracy.

So we start the project by first installing the necessary libraries. We first install tensorflow to ensure that the dataset is available and is loaded for training the model.

```
!pip install tensorflow tensorflow-datasets
```

Next we load the dataset from TFDS (tensor-flow dataset) which provides ready to use dataset splits. Printing counts helps us confirm dataset size and to check how many images are available.

```
#Loading the images

import tensorflow_datasets as tfds


dataset, info = tfds.load("cycle_gan/horse2zebra", with_info=True, as_supervised=True)

bike_ds = dataset["trainB"]   # This contains zebra images in the horse2zebra dataset


print("Total zebra images:", info.splits["trainB"].num_examples)
```

In the next step, we do standardization of all images for a common input shape. Further we define how many images are to be to be processed together during the training through the '**batch**' method. During preprocessing we have used **resize** to ensure a consistent input dimension. We also have done **casting** to float32 and divided by 255.0 scales pixel values to [0,1], to improve numerical stability and speed up learning. We have also created mapping function to attach labels.

```
import tensorflow as tf


# Step 1 — Preprocessing and labeling, combine into one dataset

IMG_SIZE = 128

BATCH = 32

AUTOTUNE = tf.data.AUTOTUNE


def preprocess_image(image):

    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
```

```
    image = tf.cast(image, tf.float32) / 255.0
    return image

def map_horse(image, _):
    return preprocess_image(image), 0   # label 0 -> horse

def map_zebra(image, _):
    return preprocess_image(image), 1   # label 1 -> zebra

horses_labeled = dataset['trainA'].map(map_horse, num_parallel_calls=AUTOTUNE)
zebras_labeled = dataset['trainB'].map(map_zebra, num_parallel_calls=AUTOTUNE)
```

In the next step, we took the two separate image sets from the Horse2Zebra dataset—**horse images** (trainA) and **zebra images** (trainB)—and converted them into a single labelled dataset suitable for supervised learning. Here, we labelled **Horse images as 0 and Zebra images as 1.** Further, the datasets were merged into one dataset using **concatenate**, which created one single large dataset for all the horse and zebra images with their correct labels.

```
# Combine datasets
combined = horses_labeled.concatenate(zebras_labeled)
combined = combined.shuffle(2000, seed=42)
```

After this, we created the training dataset and validation dataset. Here we allocated 20% of the data for validation purpose. We did batch, which groups image to process 32 at a time for faster training. We used prefetch(AUTOTUNE), so that the model loads the next batch while the model is being training the current one. We used skip while creating the training dataset, to ignore the first lot which has been used for validation purpose.

```
# Step 2 — Training/validation split using dataset.take / skip
total = 0
for _ in combined:
    total += 1
print("Total combined examples (approx):", total)

# Note: TFDS splits often have ~1000 images per domain; adjust if needed.
val_frac = 0.2
val_size = int(total * val_frac)

val_ds = combined.take(val_size).batch(BATCH).prefetch(AUTOTUNE)
train_ds = combined.skip(val_size).batch(BATCH).prefetch(AUTOTUNE)

print("Train batches:", tf.data.experimental.cardinality(train_ds).numpy(),
      "Val batches:", tf.data.experimental.cardinality(val_ds).numpy())
```

Next step, which is Step 3, we build the CNN model which classifies whether the image is of horse or zebra. After importing the Keras modules, which provides access to CNN layers, we build a sequential CNN model. We defined the input size which provides the model what shape of images it will have. We also added the convolution and maxpooling layers.The convolution layer extracts the features from the images like edges, pattern and helps the model to learn more complex features. Maxpooling reduces overfitting and reduces spatial size of the feature maps. We have used **Flatten**, to convert the extracted feature map to 1D. Adding **Dense Layers,** makes model learn high-level representations for classification.

```
# Step 3 — Building a small CNN classifier
from tensorflow.keras import layers, models

model = models.Sequential([
```

```
    layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPool2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPool2D(),
    layers.Conv2D(128, 3, activation="relu"),
    layers.MaxPool2D(),
    layers.Flatten(),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.4),
    layers.Dense(1, activation="sigmoid")   # binary output
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

In next step, i.e. Step 4, we feed the training data into CNN for it to learn patterns and monitor the performance on validation data.

Here, we set **epoch** to 12. Here, we used **validation_data=val_ds ,** which allows to track how well the model performs on unseen images thereby preventing overfitting.

```
# Step 4— Training the model
EPOCHS = 12
```

```
history = model.fit(
    train_ds,
    epochs=EPOCHS,
    validation_data=val_ds
)
```

In next step, which is Step 5, we plot the validation and accuracy of the model.

```
# Step 5 — Plotting
import matplotlib.pyplot as plt

plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.title('Accuracy')
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.title('Loss')
plt.legend()
plt.show()
```

With the plot so obtained we can say that both the training accuracy and validation accuracy increase smoothly as training accuracy starts around **0.68** and gradually increases close to **0.98** while validation accuracy also increases and stays very close to training accuracy, reaching **0.96–0.98** by the end. This shows that the model is learning well. As the accuracy is high, there is no sign of underfitting. As the training loss is dropping, this shows that model is converging properly. Since there is no divergence between training and validation loss, it shows that there is no overfitting.

Now in Step 6, we took a small batch of validation images and asked the model to predict their classes, converting the predictions **into 0 and 1 labels**, and displaying the images along with their correct and predicted labels.

#Step 6 Visualizing

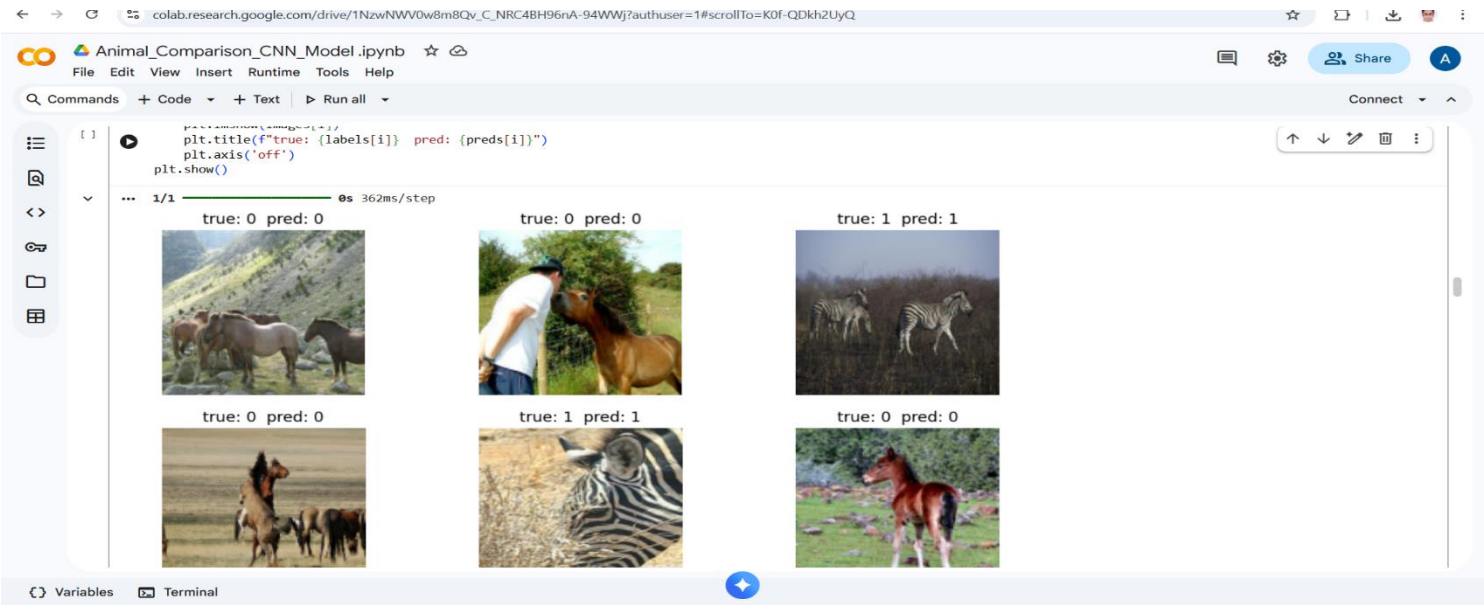
import numpy as np

import matplotlib.pyplot as plt

 # get one batch from validation

for images, labels in val_ds.take(1):

```
    preds = model.predict(images)
    preds = (preds.flatten() > 0.5).astype(int)
    images = images.numpy()
    labels = labels.numpy()
    break

# show 6 examples
plt.figure(figsize=(12,6))
for i in range(6):
    ax = plt.subplot(2,3,i+1)
    plt.imshow(images[i])
    plt.title(f"true: {labels[i]}  pred: {preds[i]}")
    plt.axis('off')
plt.show()
```



The prediction we get from visualization shows that the model correctly classifies all tested images from the validation set. Despite the differences, the model accurately distinguished between horses and zebras indicating that the CNN has successfully learned important features and generalizes well to unseen images.

Next step, which is Data Augmentation, we applied augmentation to the dataset. This takes one image at a time and applies augmentation transformation to only the image and returns the augmented image along with the label. This improves model robustness and reduces overfitting.

```
#Data augmentation

data_augmentation = tf.keras.Sequential([

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(0.1),

    layers.RandomZoom(0.1),

    layers.RandomContrast(0.2),

])
```

In next step, which is modifying the training, we change the training process so that the model now trains on **augmented images** instead of the original dataset thereby improving generalization and prevent overfitting. In this, each training batch now passes through data augmentation layers that apply random flips, rotations, zooms, and contrast changes. The augmented dataset is then prefetched for efficiency. The model is retrained using the augmented data, which helps increase robustness, reduce overfitting, and improve overall performance.

```
#Modifying training

def augment(image, label):

    image = data_augmentation(image)

    return image, label


train_ds_aug = train_ds.map(lambda x, y: augment(x, y),

                            num_parallel_calls=AUTOTUNE)

train_ds_aug = train_ds_aug.prefetch(AUTOTUNE)
```

After modification, we train the model. The model was retrained using the augmented dataset **(train_ds_aug)** for 12 epochs. The augmentation which was done, introduced variations in the images, helping the model become more robust to changes in orientation, brightness, scale, and contrast. The validation data however remained unchanged to ensure fair evaluation.

```
#Training
history = model.fit(
    train_ds_aug,
    epochs=EPOCHS,
    validation_data=val_ds
)
```

After the training step, we load the libraries and load the dataset. Once done, we perform the data augmentation and preprocessing. After this we do LR Scheduler and Early-stopping. In this step, we optimize and stabilize training by using callbacks. The LR Scheduler monitors validation loss while Early-Stopping stops training if validation loss has NOT improved for **4 epochs thereby restoring the best weights.**

Here, we have made epochs to 20, but training will stop once validation stops improving.

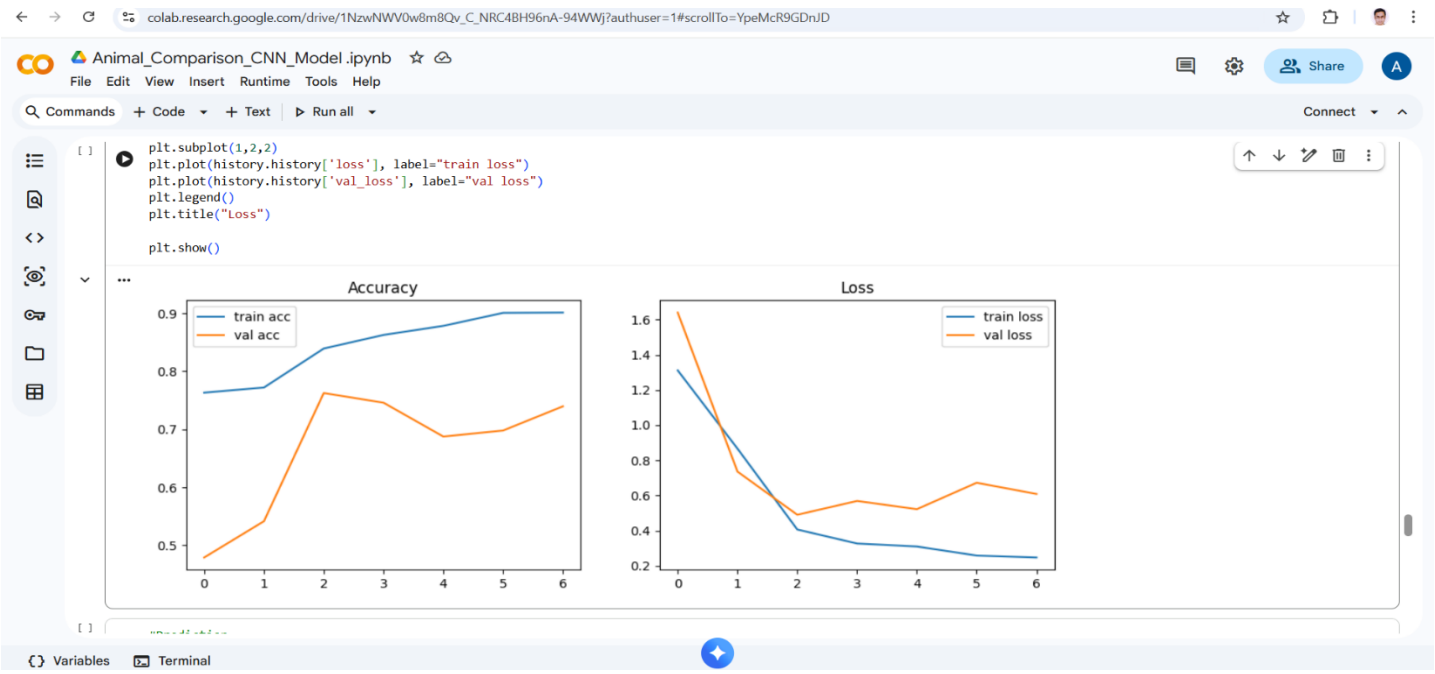
```
#LR Scheduler
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    patience=2,
    factor=0.5,
    min_lr=1e-6
)

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=4,
    restore_best_weights=True
```

)

Once the model is trained after LR Scheduling, we move to next step which is Visualization.

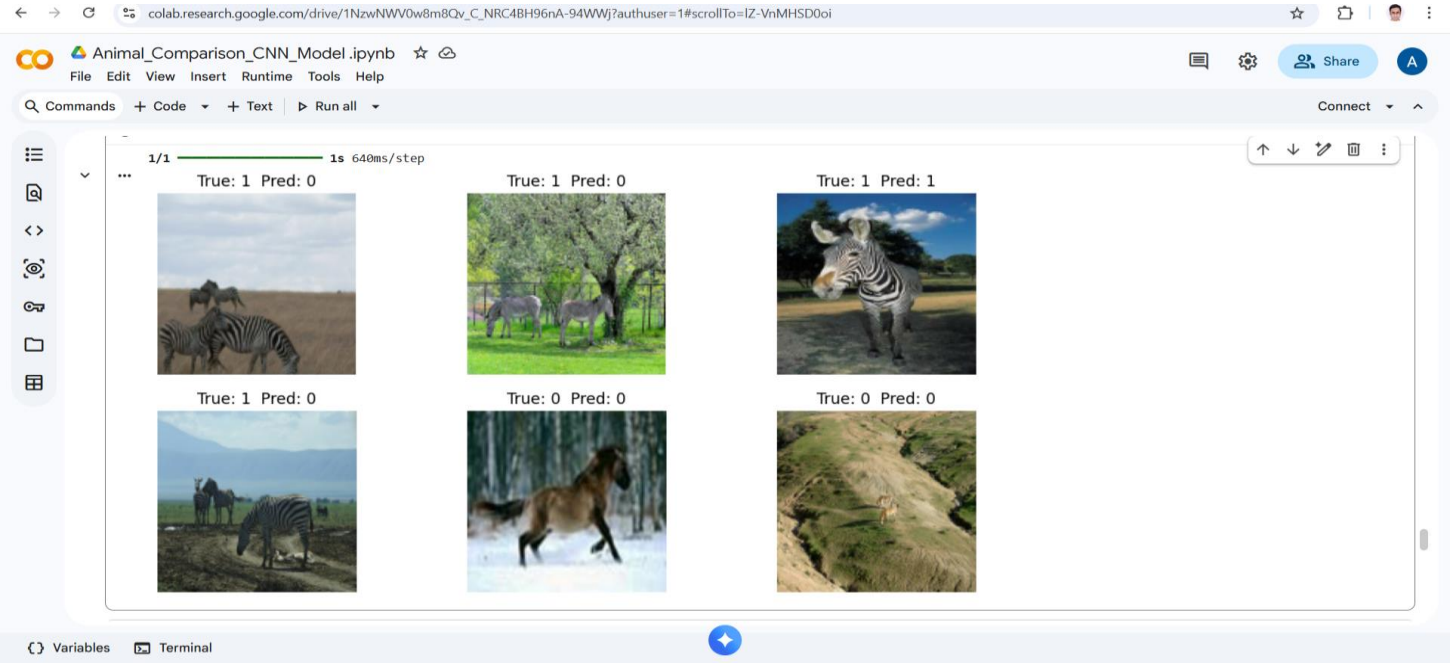
This plot shows how training accuracy improved over each epoch and whether the model is overfitting or underfitting. The graphs help evaluate model performance and confirm whether the augmentations and training modifications improved the results.

```
#Visualization

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)

plt.plot(history.history['accuracy'], label="train acc")

plt.plot(history.history['val_accuracy'], label="val acc")

plt.legend()

plt.title("Accuracy")


plt.subplot(1,2,2)

plt.plot(history.history['loss'], label="train loss")

plt.plot(history.history['val_loss'], label="val loss")

plt.legend()

plt.title("Loss")


plt.show()
```



The graph shows that the model is learning effectively on the training data, However the validation accuracy is low and fluctuating. Although we have introduced Early Stopping and LR Scheduler, the fluctuation is due to small data size, suggesting that the model's generalization ability is limited at this stage.

After this, we move to the next step, which is prediction. Here, we evaluate the model by making predictions on a batch of validation images and visualizing them. The result is that model successfully identified both horses and zebras, demonstrating strong generalization and the ability to correctly classify images.

```
#Prediction

for images, labels in val_ds.take(1):

    preds = model.predict(images)

    preds = (preds.flatten() > 0.5).astype(int)

    break


plt.figure(figsize=(12,6))

for i in range(6):

    plt.subplot(2,3,i+1)

    plt.imshow(images[i])
```

```
plt.title(f"True: {labels[i].numpy()}  Pred: {preds[i]}")
plt.axis("off")

plt.show()
```